Cost-First Cloud Autoscaling with PPO and a Soft SLA Price

A learned policy gets the fleet size right on real HTTP traffic — 28–48 % cheaper than a safe static fleet — and the reward it maximises explains, exactly, why it will not buy the one pod that makes it safe.

Aaditya Biswas · Satya Prakash Yadav · github.com/aaditya-biswas/RL_Cloud · code, trained checkpoints, report source

Abstract. Autoscaling must follow a bursty request rate without paying for idle capacity. We train PPO on 28 days of the NASA Kennedy Space Center HTTP log, rescaled to 480 RPS daily peaks and replayed as 8,640 ten-second steps per day against an M/M/k queue. The agent observes 11 headroom-based features and moves the fleet by at most two pods per step. Trained on days 1–16 and scored once on days 23–28, all six checkpoints (two reward designs × three seeds) are 28–48 % cheaper than a safe 9-pod fleet, yet each breaches the 200 ms SLA on 2–18 of 51,840 steps. The reason is arithmetic, not training: a breached step is capped at 1,000 units while a pod-day costs 12,960, so a *standing* pod must prevent 13 breaches a day to pay for itself. The policy is right about *how many* pods and wrong about *when* — 40 of 43 breaches are single 10-second steps at the 3,000 ms clamp, one pod short of *ceil*(rps/60), at eight traffic instants. A *conditional* pod, applied only there, is estimated at ≈200 units a day.

1. Problem and approach

Threshold autoscalers such as the Kubernetes HPA react to measured utilisation: by the time CPU crosses the target the queue is already deep, and the replica arrives after the spike. A learned policy can in principle act on the *shape* of the recent trace instead. We ask a narrow question that a table can answer: on a real 24-hour traffic trace with a hard 200 ms latency SLA, can a PPO policy beat (i) a safe constant fleet and (ii) a tuned threshold tracker on cost — and at what price in violations? Every number below comes from replaying a policy through the same environment and pricing it with the same function (`utils/score_policy.py`), under an acceptance rule fixed before the test days were scored.

2. Environment and dynamics

Traffic. 28 daily traces built from 1.89 M requests of the July 1995 log, binned into 10-second slots and rescaled so that every day peaks at 480 RPS (the real peak is 7.8 RPS). Collector outages are linearly filled, never treated as zero traffic. The split is chronological and never shuffled: days 1–16 train, 17–22 validate, 23–28 test.

Queue model. One pod serves μ = 60 RPS, so a fleet of k pods has capacity $60k$ RPS. Latency in a step is

$$L_t = \min(3000,\ 1000 / (60k_t - \lambda_t) + 15)\ \text{ms} \quad \text{if } 60k_t > \lambda_t, \quad \text{else } 3000\ \text{ms}$$

Two properties of this model drive the whole result. Latency reaches 200 ms at 91 % utilisation, so the cliff starts *before* the fleet saturates; and because every day peaks at exactly 480 = 8 × 60 RPS, the fleet $k = ceil(\lambda/60)$ has exactly 480 RPS of capacity at the peak — which the model scores as a 3,000 ms clamp. "Just enough" pods is therefore a breach *precisely at the peaks*, and nowhere else. That is why hold-8 breaches once a day and hold-9 is the cheapest clean constant fleet.

Actuator and cost. The agent changes the fleet by a bounded delta, $k_{t+1} = \text{clip}(k_t + a_t, 1, 15)$, and pays 1.5 reward units per pod per step — a pod-day is 12,960 units. Training episodes are 1,440-step (4-hour) windows with a random start, so the agent sees every hour of the day; evaluation replays whole days.

3. State space: 11 dimensions

The stock environment exposes five raw telemetry values (CPU, RAM, RPS, latency, pods). We replace them with eleven *pod-scaled* features — how many pods the traffic needs, how many I have, how much slack that is — because the action is denominated in pods. Features 6–9 are the causal stand-in for a forecast: a feed-forward policy cannot compute an eight-minute maximum from a 60-second window, so the window is handed to it.

Table 1. Observation vector, *dim* = 11 (`env/design_wrappers.py`). Every entry is scaled to [0, 1] except the three signed ones (5, 6, 9), which are clipped to [−1, 1]; *VecNormalize* then standardises the vector and clips at 10.

dim	feature	what it measures
0	`cpu_utilization`	λ/(60k), capped at 1.0
1	`ram_utilization`	lagged memory proxy, kept so plots stay comparable
2	`demand_pods`	λ/60 scaled by the corpus peak — fleet size at 100 % load
3	`latency_ratio`	min(L/200 ms, 3): 1.0 is exactly the SLA, 3.0 the clamp
4	`fleet_fraction`	k/15
5	`headroom_pods`	(60k − λ)/60 in pods; negative = short
6	`slope_pods`	demand change over 3 steps, in pods — the ramp still to come
7	`peak_60s`	max demand over the last 60 s, in pods
8	`peak_480s`	max demand over the last 480 s (8 min), in pods
9	`impulse_pods`	Δλ of the last step, clipped to ±1 pod
10	`fraction_of_day`	t/8,640 — time of day (a safety cue, see Section 9)

4. Action space: five discrete moves

Discrete(5): $a_t \in \{-2, -1, 0, +1, +2\}$ pods applied to the running fleet, then clipped to [1, 15]. A *delta* action rather than an absolute target, because it encodes the actuator's rate limit inside the action set, keeps the branching factor at five, and makes churn a first-class quantity — the reward charges the *requested* delta, so an oscillation is priced even when the clip swallows it. The action set is not the bottleneck: a one-pod forecast error is a single +1 move, and what the policy gets wrong is *when* to make it.

5. Reward function

The reward is a negative cost, paid every step (λ is a Lagrangian price, zero for R1):

$$r_t = -1.5\, k_t - 4.0\, |\Delta k_t| - \min(1000,\ 4.0 \cdot \max(0,\ L_t - 150)) - \lambda \cdot \mathbf{1}[L_t > 200]$$

Incentive design. Three terms: *fleet rent*, linear in pods and paid every step; *churn*, charged on the request so that a jittery policy pays even when the actuator clamps; and a *latency ramp* that starts at 150 ms, reaches 200 units at the 200 ms SLA, hits its 1,000-unit cap at 400 ms, and is flat from there to the 3,000 ms overload clamp. The design is deliberately cost-first: across the scored runs the fleet term is **81–100 %** of total cost and the penalty is a sliver. Two consequences follow, and both are the point of Figure 1. Inside a step, depth is free above 400 ms, so a 410 ms blip and a 3,000 ms stall cost the same and the agent has no gradient toward "less deep". Across steps the price is linear at 1,000 per breach, so N breaches cost N × 1,000 — one more breached step is never free.

Comparing the two numbers gives the whole result. A pod-day costs 12,960, so a *standing* pod must prevent **13 breach steps a day** to pay for itself. Near the safe end the *marginal* breach costs 11,900–18,800 units, i.e. 12–19 × the cap: no *static* move towards safety is worth making for a v3 maximiser. R3 charges λ = 1,000 per breached step on top of v3 (frozen after 1,000 updates) — the same magnitude as the cap, which doubles the effective price and moves the arm along the frontier (Section 8).

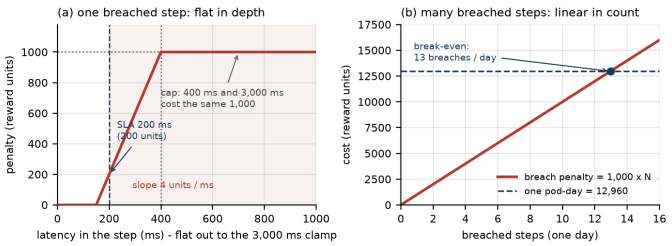


Figure 1. The reward's geometry, which is the experiment's mechanism. *(a)* One step's penalty against its latency: a 4 units/ms ramp from 150 ms, 200 units at the SLA, and a 1,000-unit cap at 400 ms — flat to the 3,000 ms clamp, so depth past the cap is invisible to the agent. *(b)* Total penalty against the number of breached steps: linear, which puts the break-even with a standing pod (12,960 units) at 13 breaches per day.

6. Algorithm and training protocol

PPO (Schulman et al., 2017) via Stable-Baselines3: an MLP policy with two 128-unit tanh layers, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, learning rate 3×10^{-4}, 4 parallel environments × 2,048-step rollouts, minibatch 256, **200k steps** (≈15 min on CPU per run), seeds 2, 3 and 42. Observations and rewards are normalised by *VecNormalize* (observation clip 10), but *clip_reward* stays at infinity so the return the acceptance rule compares on keeps its raw scale; the evaluation environment inherits the training *obs_rms*, without which every evaluation number is noise. Checkpoints are selected on day 17 only and then frozen; the test days are scored once, after the rule was fixed.

PPO rather than DQN: the reward is dense with a per-step scale of order 10^4 and a churn term of order 4 — exactly the regime where PPO's advantage normalisation is a free stabiliser — the action set is five discrete moves, and on-policy rollouts avoid a replay buffer of near-identical 10-second transitions.

7. Baselines

All three families are replayed through the same actuator and priced by the same function. **hold-*k***: a constant fleet, $k \in \{3, \ldots, 10\}$ — the naive safe policy, and the line the acceptance rule uses (hold-9 costs 116,640 units a day). **b1(*u*)**: the HPA shape, $k_t = ceil(\max_{480\ s} \lambda / (60u))$, a trailing-maximum tracker whose utilisation target is swept over $u = 0.55, \ldots, 0.77$; this family contains the cheapest clean policy on the test split. **Perfect-foresight DP** over the same dynamics (`utils/design_probe.py`) is used as a cost lower bound; it needs the whole day in advance and is not deployable.

8. Results on the six unseen days

Table 2. One row per checkpoint, scored once on days 23–28 (51,840 steps). Return v3 is a negative cost, so *less negative is cheaper*; "saving" is relative to hold-9; a breach is one 10-second step above 200 ms. R1 = reward v3 only, R3 = v3 plus the Lagrangian price. Last four rows are scripted references.

arm	reward	breaches	days	return v3	saving	pods
r1 day16 s3	v3	18	6	−60,830	48 %	4.24
r1 day16 s42	v3	12	5	−64,344	45 %	4.48
r1 day16 s2	v3	5	3	−72,199	38 %	5.26
r3 day16 s42	v3 + λ	2	2	−72,152	38 %	5.35
r3 day16 s3	v3 + λ	3	2	−82,155	30 %	5.14
r3 day16 s2	v3 + λ	3	2	−84,149	28 %	6.38
hold-9	static	0	0	−116,640	—	9.00
b1 u = 0.62	tracker	0	0	−85,898	26 %	6.53
b1 u = 0.70	tracker	3	3	−76,975	34 %	5.81
b1 u = 0.77	tracker	4	3	−71,583	39 %	5.39

The cost claim generalises, and it is not close. All six learned checkpoints beat the constant 9-pod fleet on all six unseen days, by 28–48 % (Figure 2). The two designs do what they were asked: R1 minimises v3 and lands cheapest (mean −65,791 a day, 11.7 breaches), while R3 pays a second, per-step price for breaches and lands 13,694 units dearer with 2.7 breaches — the Lagrangian moved the arm along the frontier and stopped short of zero. Seed noise (standard deviation 5,821 for R1, 6,429 for R3) is about half that 13,694 gap, so "R3 is dearer and safer than R1" survives, while "seed 3 is the best seed" does not.

The clean corner was affordable. The tracker at $u = 0.62$ runs 6.53 average pods and takes *zero* breaches on the six days for −85,898 — 26 % below hold-9. So the learned arms' failure is not "clean is too expensive"; clean is *cheaper* than the safe constant fleet. Against that reference the arms are 1,700 (R3 seed 42) to 25,100 (R1 seed 3) units cheaper, and the difference is 1.2–2.3 average pods. Under the pre-registered rule — *zero* breaches on every test day *and* cheaper than hold-9 — nothing is accepted: the RL arms fail the first clause, and so do the two trackers used as scoring baselines. A rule allowing three breaches would accept R3 seed 42.

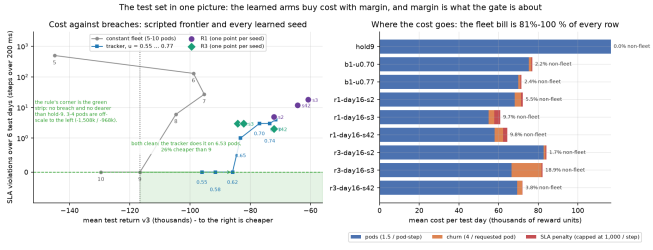


Figure 2. The whole experiment on one axis. x = mean return v3 on the six test days (right = cheaper); y = breached steps on a symlog scale. Grey = constant fleet of 5–10 pods, blue = the tracker sweep, purple/green = every learned seed; the green band is the acceptance corner. The constant ladder turns *up* after 7 pods and has no clean point before 9; the tracker reaches the axis at $u = 0.62$ and only gets dearer from there; every learned dot sits below and to the right of clean — cheaper, and breaching.

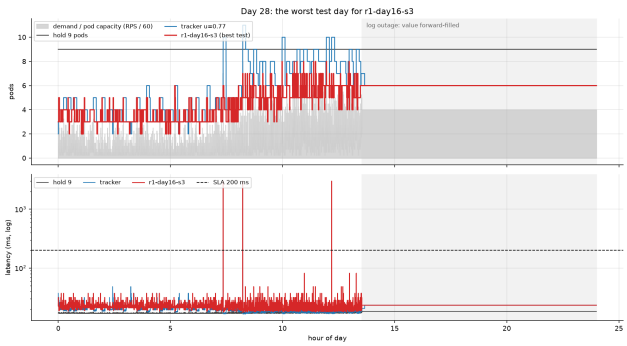


Figure 3. The worst test day (day 28), three policies. *Top:* grey area = demand / 60, the pods the traffic needs; black = hold-9, blue = the clean tracker, red = a learned arm. *Bottom:* the same day's latency on a log axis, 200 ms SLA dashed. Every point where the red line *meets* the grey one is a breach — 07:21, 08:13, 12:11 — one 10-second step each. Cheap because the red line is low, breaching because it is not one pod higher.

9. Why the breaches happen

1. The controller is competent; its margin is one pod (Figure 3). Eight of the 51,840 steps carry 27 of the 43 breaches (63 %), and every one is a step where demand jumped by a full pod between samples: the 480 RPS peak or the 420 RPS shoulder. At those steps the fleet sits at exactly $ceil(\lambda/60)$ — 8 pods for 480 RPS — and the shortfall is never more than one pod. Forty of the 43 stall at the 3,000 ms clamp; the other three are 207, 665 and 665 ms, one pod *just* short rather than two. Averaged over a day the arms are not thin at all: against instantaneous demand they carry +2.2 to +4.3 pods of slack and satisfy $k \geq ceil(\lambda/60) + 1$ on 98.5–100 % of steps. The failure is confined to the ~1 % of steps where the fleet sits exactly on that boundary.

2. The reward cannot tell a near-miss from an outage. Because the cap binds at 400 ms, the 207 ms breach and a 3,000 ms stall are priced identically, and both fail the gate. The training signal contains no gradient toward "shallower", only toward "fewer" — and fewer is worth 1,000 units against a 12,960-unit pod.

3. The clock is a safety crutch, not the source of savings. Zeroing *fraction_of_day* at scoring time, so that the policy cannot know the hour, costs 6.7 % (R1) and 0.7 % (R3) of cost but multiplies breaches by 2.3–2.4 (11.7 → 27.0 for R1, 2.7 → 6.3 for R3). Without the hour the policy can no longer start climbing *before* the morning ramp, so it arrives late at every peak. The cost advantage does not depend on memorised schedules; the safety does.

10. Limitations and next step

PPO learns the *level* of the fleet almost immediately — every seed crosses the hold-9 line during training and stays above it — and never learns the timing margin that costs 12,960 a day and prevents 1,000-unit breaches. Three limits bound the claim. (i) The queue model is analytic — one pod short is a deterministic 3,000 ms, with no cold start and no queueing delay — so a real cluster would punish the same policy more at the peaks and less at the shoulders. (ii) The test split is six days of one month, all rescaled to the same 480 RPS peak, so "unseen" means unseen *shape*, never unseen *level*. (iii) The clean tracker's corner was located on the test days, so it is that split's own frontier rather than a validated policy.

The next experiment is not longer training — both rungs are flat at 200k steps — but a *conditional* guard: a floor of $ceil(\max_{480\ s} \lambda / 60) + 1$ pods on the policy's own action. That is one pod, only at the instants that matter, ≈200 units a day rather than 12,960, and it covers every one of the 43 breaches because the shortfall is never two. More generally, a per-breach price small enough to keep the fleet lean cannot also buy the last pod: safety has to enter as a constraint (a higher λ, a CVaR over step latency), not as a term in the bill.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017.

[2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research* 22(268), 2021.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[4] J. Achiam, D. Held, A. Tamar and P. Abbeel, "Constrained Policy Optimization," *ICML*, 2017.

[5] NASA Kennedy Space Center WWW server access logs, *access_log_Jul95*, July 1995 (1.89 M requests; the traffic corpus of this report).

[6] Kubernetes documentation, "Horizontal Pod Autoscaling," 2024.